We'll skip the last step—translating the commands from the pseudo language into AWS API calls. You've already learned everything you need to know about infrastructure as code: it's all about dependencies. Let's apply those newly learned ideas in practice.

## 4.5 Using AWS CloudFormation to start a virtual machine

In the previous section, we created JIML to introduce the concept of infrastructure as code. Luckily, AWS already offers a tool that does much better than our JIML: *AWS CloudFormation*.

**NOTE** When using CloudFormation, what we have been referring to as a blueprint so far is actually referred to as a CloudFormation template.

A *template* is a description of your infrastructure, written in JSON or YAML, that can be interpreted by CloudFormation. The idea of describing something rather than listing the necessary actions is called a *declarative approach*. Declarative means you tell CloudFormation how your infrastructure should look. You aren't telling CloudFormation what actions are needed to create that infrastructure, and you don't specify the sequence in which the actions need to be executed. The benefits of CloudFormation follow:

- *It's a consistent way to describe infrastructure on AWS.* If you use scripts to create your infrastructure, everyone will solve the same problem differently. This is a hurdle for new developers and operators trying to understand what the code is doing. CloudFormation templates provide a clear language for defining infrastructure.